

FULL STACK DEVELOPMENT

FASE 5 | AULA 04 -

BD COM MICROSSERVIÇOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS.....	17
O QUE VOCÊ VIU NESTA AULA?	19
REFERÊNCIAS	20

EXEMPLO

O QUE VEM POR AÍ?

Nesta aula, falaremos sobre como a persistência de dados em um ambiente de microsserviços apresenta desafios únicos, como a necessidade de isolar dados por serviço, manter a consistência distribuída e gerenciar transações complexas. Vamos discutir a diferença entre o uso de bancos de dados por serviço e bancos de dados compartilhados, além de explorarmos padrões de design como Event Sourcing e CQRS, que ajudam a manter a consistência e integridade dos dados em sistemas distribuídos. Também veremos as diferenças entre bancos de dados SQL e NoSQL, incluindo suas características, casos de uso e ferramentas associadas.

Resumindo, com essa aula você poderá:

- Entender as complexidades de gerenciar dados em microsserviços, incluindo isolamento de dados, consistência distribuída e transações distribuídas (desafios de persistência).
- Explorar padrões como Event Sourcing e CQRS, que facilitam a gestão de dados e a manutenção da consistência em sistemas distribuídos (padrões de design).
- Comparar bancos de dados SQL e NoSQL, suas características e quando usar cada um, além de conhecer ferramentas específicas para cada tipo de banco de dados.
- Ver exemplos de integração e casos práticos de como empresas líderes de mercado implementam bancos de dados em suas arquiteturas de microsserviços.

Compreender esses conceitos é crucial para projetar sistemas que sejam escaláveis, resilientes e capazes de lidar com a complexidade da gestão de dados em um ambiente de microsserviços.

HANDS ON

Nesta aula prática, os principais desafios e soluções para gerenciar bancos de dados em arquiteturas de microsserviços serão apresentados. Haverá uma discussão sobre os motivadores para escolher entre bancos de dados SQL e NoSQL, destacando a importância de selecionar a tecnologia adequada para cada serviço. Além disso, vamos explorar padrões de design como Event Sourcing e CQRS, que ajudam a manter a consistência e integridade dos dados em sistemas distribuídos.



SAIBA MAIS

Desafios de persistência

Os bancos de dados em arquiteturas de microsserviços apresentam desafios únicos em termos de persistência de dados. Diferente das arquiteturas monolíticas, onde um único banco de dados pode ser usado para toda a aplicação, cada microsserviço pode ter seu próprio banco de dados. Essa abordagem traz benefícios em termos de autonomia e escalabilidade dos serviços, mas também apresenta desafios significativos.

Nos sistemas monolíticos tradicionais, um único banco de dados centralizado facilita a gestão da consistência e a execução de transações ACID, garantindo que todas as operações sejam atômicas e que os dados permaneçam consistentes. No entanto, essa centralização pode se tornar um gargalo conforme a aplicação cresce, dificultando a escalabilidade e a flexibilidade do sistema. A migração para uma arquitetura de microsserviços busca resolver esses problemas, mas introduz uma nova camada de complexidade na gestão de dados.

Em uma arquitetura de microsserviços, a independência dos serviços é um dos principais benefícios, permitindo que equipes diferentes desenvolvam, escalem e implantem serviços de forma autônoma. Cada serviço pode escolher a tecnologia de banco de dados que melhor atende às suas necessidades específicas, seja um banco relacional, como PostgreSQL, ou uma solução NoSQL, como MongoDB. Contudo, essa independência também significa que os serviços devem encontrar maneiras eficazes de se comunicar e manter a consistência dos dados em um ambiente distribuído. A coordenação e a sincronização de dados entre serviços tornam-se críticas para o funcionamento do sistema como um todo.

Desafios comuns

Isolamento de dados

Em um sistema de microsserviços, cada serviço é responsável por seu próprio conjunto de dados. Logo, um serviço não deve acessar diretamente os bancos de

dados de outros serviços. Esse isolamento promove a autonomia dos serviços, permitindo que cada equipe de desenvolvimento escolha a tecnologia de banco de dados que melhor se adapta às suas necessidades específicas. No entanto, isso também leva a desafios, como a sincronização de dados e a possível duplicação de informações.

Para contornar esses problemas, os serviços frequentemente se comunicam através de APIs ou eventos. Por exemplo, um serviço de pedidos pode emitir um evento "pedido criado" que é consumido por um serviço de inventário para ajustar os níveis de estoque. Embora essa abordagem facilite a independência dos serviços, ela também requer um planejamento cuidadoso para garantir que os dados estejam sincronizados e atualizados em toda a aplicação.

Consistência distribuída

Manter a consistência dos dados em um ambiente distribuído é um dos maiores desafios nas arquiteturas de microsserviços. Em um sistema monolítico, a consistência pode ser garantida de forma mais direta por transações ACID (Atomicidade, Consistência, Isolamento, Durabilidade). No entanto, em microsserviços, a garantia de consistência forte pode comprometer a disponibilidade e a escalabilidade do sistema.

Por conta disso, muitas arquiteturas de microsserviços adotam a consistência eventual. Nesse modelo, os dados podem estar temporariamente inconsistentes, mas eventualmente, após um certo período, se tornam consistentes. Isso melhora a disponibilidade e a escalabilidade, mas requer que os sistemas sejam projetados para lidar com a eventual inconsistência de maneira eficiente e transparente para os usuários.

Transações distribuídas

Realizar transações que envolvem múltiplos serviços requer coordenação complexa. Um modelo comum é o de transações distribuídas, onde uma transação abrange vários serviços diferentes. Um exemplo disso é a abordagem de sagas.

Nas sagas, cada serviço executa uma transação local e publica um evento ou mensagem indicando o sucesso ou a falha da operação. Se uma parte da transação

falhar, serviços subsequentes devem executar ações compensatórias para reverter as mudanças realizadas até aquele ponto. Por exemplo, em um sistema de reserva de viagens, se a reserva de um voo falhar, as reservas de hotel e carro alugado também devem ser canceladas para manter a consistência do sistema.

Gerenciamento de schemas

Cada serviço em uma arquitetura de microsserviços pode evoluir seu esquema de dados de maneira independente, e isso pode levar a incompatibilidades e dificuldades na integração. É crucial manter um gerenciamento de versão robusto para os esquemas de dados e garantir que as mudanças sejam compatíveis com versões anteriores.

Ferramentas de migração de banco de dados, como Flyway ou Liquibase, são frequentemente utilizadas para gerenciar mudanças no esquema de dados de forma segura e controlada. Elas permitem que as alterações no esquema sejam versionadas e aplicadas de maneira incremental, minimizando o risco de incompatibilidades e facilitando a recuperação em caso de problemas.

Para mitigar esses desafios, é importante adotar boas práticas de design e comunicação entre equipes, além de investir em ferramentas de monitoramento e automação que ajudem a manter a integridade e a consistência dos dados em todo o sistema distribuído.

Consistência e integridade dos dados

Em um ambiente de microsserviços, garantir a consistência e a integridade dos dados é um dos maiores desafios. Diferente das arquiteturas monolíticas, as arquiteturas de microsserviços precisam lidar com a distribuição dos dados em múltiplos serviços e bancos de dados. Posteriormente, exploraremos os principais conceitos e padrões que ajudam a manter a consistência e a integridade dos dados em ambientes distribuídos.

Consistência eventual

Em um ambiente distribuído, é comum adotar a consistência eventual, onde os dados são eventualmente consistentes, mas podem estar temporariamente

inconsistentes. Esse modelo melhora a disponibilidade e a escalabilidade, mas requer mecanismos para lidar com a eventual inconsistência.

Vantagens

- **Disponibilidade:** serviços permanecem altamente disponíveis, mesmo durante picos de carga.
- **Escalabilidade:** facilita a escalabilidade horizontal, permitindo que diferentes serviços processem suas operações de forma independente.

Desafios

- **Complexidade de implementação:** requer um planejamento cuidadoso e a implementação de mecanismos de resolução de inconsistências.
- **Transparência para o usuário:** deve ser garantido que os usuários não percebam as inconsistências temporárias, ou que estas sejam resolvidas de maneira que não causem problemas de usabilidade.

Por exemplo, em um sistema de e-commerce, um item pode temporariamente aparecer disponível para dois usuários diferentes, mas apenas um poderá realmente comprá-lo. Os sistemas devem ser projetados para lidar com essas inconsistências temporárias de maneira transparente para o usuário, e isso pode envolver a implementação de mecanismos de compensação ou a comunicação clara aos usuários sobre o estado de suas transações.

Padrões de transação

Sagas

Uma abordagem para gerenciar transações distribuídas é o padrão de Sagas. Nesse padrão, cada passo de uma transação é gerenciado por um serviço separado e a coordenação é feita por meio de mensagens. Se um passo falha, uma mensagem de compensação é enviada para desfazer as ações anteriores.

Vantagens

- **Isolamento de falhas:** falhas podem ser gerenciadas de forma localizada, sem afetar o sistema como um todo.
- **Escalabilidade:** cada serviço pode escalar de forma independente, gerenciando suas próprias transações.

Desafios

- **Complexidade de coordenação:** requer coordenação complexa entre serviços para garantir que as transações sejam completadas ou revertidas de maneira adequada.
- **Latência:** a latência pode ser introduzida devido à comunicação assíncrona entre serviços.

Two-Phase Commit

O protocolo Two-Phase Commit garante que todas as partes de uma transação concordem antes de confirmá-la, mas pode introduzir latência e complexidade. Este protocolo envolve duas fases: a fase de preparação, onde todos os serviços envolvidos se preparam para a transação e confirmam que podem concluí-la, seguida de uma fase de commit, onde a transação é finalizada.

Vantagens

- **Consistência forte:** garante que todas as partes da transação concordem antes de finalizá-la, mantendo a consistência dos dados.
- **Segurança:** minimiza o risco de dados inconsistentes ao garantir que todos os serviços estejam prontos antes de realizar o commit.

Desafios

- **Latência:** pode introduzir latência significativa, especialmente em sistemas com muitos serviços.
- **Complexidade de implementação:** requer uma infraestrutura robusta para gerenciar a coordenação entre serviços.

Pensemos em um sistema bancário: uma transferência bancária entre diferentes contas pode usar o Two-Phase Commit para garantir que os débitos e créditos sejam realizados apenas se todas as partes da transação puderem ser concluídas com sucesso, evitando inconsistências no saldo das contas.

Padrões de design

Event Sourcing

Event Sourcing é um padrão de design que transforma a maneira como os dados são armazenados e gerenciados. Em vez de manter o estado atual diretamente em tabelas, todos os eventos que causam mudanças no estado do sistema são registrados em uma sequência de eventos. O estado atual é reconstruído ao reexecutar esses eventos, e cada evento é imutável e representa uma mudança que ocorreu, como a criação de um pedido ou a atualização de um perfil de usuário.

Essa abordagem proporciona uma maneira detalhada de rastrear todas as alterações que ocorreram no sistema, oferecendo uma visão completa do histórico de operações. Em um sistema de gerenciamento de pedidos, por exemplo, cada evento, tais como "pedido criado", "pedido atualizado" e "pedido enviado", é registrado em ordem cronológica. O estado atual do pedido pode ser reconstruído a qualquer momento reexecutando todos os eventos desde o início. Isso não apenas facilita a auditabilidade, mas também permite a capacidade de refazer ou desfazer operações ao reprocessar eventos específicos.

Benefícios

- **Auditabilidade:** cada mudança de estado é registrada, proporcionando um histórico completo das mudanças. Isso é particularmente útil para auditorias e conformidade regulatória, onde é necessário rastrear cada alteração e suas origens.
- **Consistência Eventual:** facilita a consistência eventual em sistemas distribuídos. Como todos os eventos são registrados em ordem, os serviços podem eventualmente convergir para um estado consistente ao processar os mesmos eventos.

- **Escalabilidade:** permite uma melhor escalabilidade ao distribuir o processamento de eventos entre vários serviços.

Desafios

- **Complexidade de implementação:** a implementação de Event Sourcing pode ser complexa, exigindo um bom design para a gestão e reprocessamento dos eventos. Requer uma compreensão profunda de como os eventos devem ser modelados e armazenados.
- **Espaço de armazenamento:** o armazenamento de todos os eventos pode consumir muito espaço, especialmente em sistemas com um alto volume de transações. Portanto, é necessário gerenciar eficientemente o armazenamento para evitar problemas de desempenho e custo.

Uma das possíveis aplicações desse padrão seria em um sistema de contabilidade, onde todas as transações financeiras são registradas como eventos. O saldo atual de uma conta é calculado reprocessando todos os eventos de depósito e retirada associados a essa conta, o que permite um rastreamento preciso e detalhado de todas as operações financeiras.

CQRS (Command Query Responsibility Segregation)

CQRS é um padrão de design que separa as operações de comando (escrita) e consulta (leitura) em diferentes modelos. Essa separação permite otimizar cada modelo para seu propósito específico. Por exemplo, um modelo de comando pode ser normalizado para garantir a integridade dos dados, enquanto um modelo de consulta pode ser desnormalizado para melhorar o desempenho de leitura.

Em sistemas tradicionais, as mesmas entidades e modelos são usados tanto para modificar dados (comandos) quanto para consultar dados (consultas). No entanto, essas operações têm requisitos diferentes e podem ser otimizadas separadamente. Por exemplo, em um sistema de e-commerce, as operações de comando, como “criar pedido”, “atualizar o status de um pedido” ou “cancelar um pedido”, podem ser modeladas para garantir a integridade e consistência dos dados.

Em contraste, as operações de consulta, como listar todos os pedidos de um cliente ou buscar detalhes de um pedido específico, podem ser otimizadas para

desempenho e velocidade, utilizando modelos desnormalizados que facilitam a recuperação rápida de informações.

Benefícios

- **Escalabilidade:** leitura e escrita podem ser escaladas de forma independente. Um serviço pode ter várias réplicas de leitura para atender a alta demanda de consultas sem impactar as operações de escrita.
- **Desempenho:** consultas podem ser otimizadas separadamente das operações de escrita, melhorando o desempenho. Por exemplo, uma consulta que precisa agregar dados de várias tabelas pode ser pré-calculada e armazenada em um formato otimizado para leitura.
- **Flexibilidade:** permite adaptar e escalar partes do sistema de acordo com necessidades específicas, sem comprometer outras partes.

Desafios

- **Complexidade de implementação:** a separação de comandos e consultas pode aumentar a complexidade do sistema, exigindo mais esforço de design e implementação. É necessário manter sincronia entre os modelos de leitura e escrita.
- **Consistência eventual:** em alguns casos, os dados de leitura podem estar temporariamente desatualizados em relação aos dados de escrita, exigindo mecanismos para lidar com essas discrepâncias. Isso pode requerer estratégias adicionais para garantir que as operações críticas estejam sempre consistentes.

Voltando ao exemplo do sistema e-commerce, o modelo de comando pode ser utilizado para processar pedidos e atualizar o inventário, enquanto o modelo de consulta pode ser otimizado para responder rapidamente a buscas de produtos e exibir catálogos de produtos aos usuários. Isso garante que as operações de consulta sejam rápidas e eficientes, enquanto as operações de comando mantêm a integridade e consistência dos dados.

Ferramentas e Tecnologias

Bancos de Dados SQL vs. NoSQL

Os bancos de dados desempenham um papel crucial na arquitetura de microsserviços, fornecendo a base para a persistência e gestão de dados. A escolha entre bancos de dados SQL e NoSQL é uma decisão fundamental que afeta a escalabilidade, flexibilidade e desempenho do sistema.

Os bancos de dados SQL (Structured Query Language) são conhecidos por sua estruturação rígida e suporte a transações ACID (Atomicidade, Consistência, Isolamento, Durabilidade). Em contraste, os bancos de dados NoSQL (Not Only SQL) oferecem flexibilidade de esquema e são projetados para escalar horizontalmente, suportando grandes volumes de dados e alta disponibilidade.

SQL

Os bancos de dados SQL, como MySQL e PostgreSQL, são baseados em um esquema rígido que define a estrutura dos dados com tabelas, colunas e tipos de dados específicos. Isso garante a integridade dos dados e facilita a validação. Um dos principais benefícios dos bancos de dados SQL é o suporte a transações ACID, que assegura que as operações sejam realizadas de forma atômica, mantendo a consistência e a durabilidade dos dados mesmo em casos de falhas. Além disso, os bancos de dados SQL suportam consultas complexas, permitindo a recuperação eficiente de dados relacionados por meio de joins e agregações. Essas características tornam os bancos de dados SQL ideais para aplicações que exigem consistência forte e integridade de dados, como sistemas financeiros e de processamento de pagamentos. As suas características incluem:

Estrutura Rígida: bancos de dados SQL utilizam esquemas rígidos que definem a estrutura dos dados (tabelas, colunas e tipos de dados). Isso garante a integridade dos dados e facilita a validação.

Transações ACID: suporte a transações que garantem Atomicidade, Consistência, Isolamento e Durabilidade. Isso é crucial para aplicações que requerem integridade de dados rigorosa, como sistemas financeiros.

Consultas complexas: SQL permite consultas complexas com joins e agregações, facilitando a recuperação de dados relacionados.

O uso de bancos de dados SQL é recomendado para aplicações que requerem consistência forte, como sistemas bancários, processamento de pagamentos e e-commerce, e para aplicações que precisam garantir a integridade e validade dos dados.

NoSQL

Os bancos de dados NoSQL, como MongoDB, Cassandra e Redis, oferecem uma abordagem diferente. Eles permitem a flexibilidade de esquema, o que significa que a estrutura dos dados pode mudar conforme necessário sem a necessidade de migrações de esquema complexas. Isso os torna ideais para aplicações que lidam com dados não estruturados ou semi-estruturados, como redes sociais e análise de big data. Além disso, os bancos de dados NoSQL são projetados para escalar horizontalmente, adicionando mais nós ao cluster para gerenciar grandes volumes de dados e altas cargas de trabalho. Muitos bancos de dados NoSQL adotam consistência eventual, permitindo maior disponibilidade e particionamento dos dados. Algumas de suas características são:

Flexibilidade de esquema: bancos de dados NoSQL permitem esquemas dinâmicos, o que significa que a estrutura dos dados pode mudar conforme necessário sem a necessidade de migrações de esquema.

Escalabilidade horizontal: capacidade de escalar horizontalmente adicionando mais nós ao cluster, o que facilita a gestão de grandes volumes de dados e alta carga de trabalho.

Consistência eventual: muitos bancos de dados NoSQL adotam consistência eventual, permitindo maior disponibilidade e particionamento.

Bancos de dados NOSQL são ideais para aplicações que precisam escalar rapidamente, como redes sociais, análise de big data e IoT, e para aplicações que lidam com dados que não se encaixam bem em um esquema relacional rígido (dados não estruturados ou semiestruturados).

Teorema CAP

O teorema CAP (Consistência, Disponibilidade e Particionamento) é um princípio fundamental que descreve as limitações de sistemas distribuídos, como os utilizados em arquiteturas de microsserviços. Segundo o teorema CAP, um sistema distribuído pode oferecer, no máximo, duas das três garantias seguintes:

- **Consistência:** todos os nós veem os mesmos dados ao mesmo tempo. Em outras palavras, uma vez que os dados são escritos, todos os nós do sistema retornam a versão mais recente dos dados.
- **Disponibilidade:** cada solicitação de um nó recebe uma resposta (sucesso ou falha), garantindo que o sistema esteja sempre operacional.
- **Particionamento:** o sistema continua a funcionar mesmo que partes dele estejam desconectadas ou falhem.

Em arquiteturas de microsserviços, escolher entre bancos de dados SQL e NoSQL frequentemente envolve fazer concessões baseadas no teorema CAP. Bancos de dados SQL tendem a priorizar consistência e disponibilidade, o que os torna adequados para sistemas que exigem transações ACID. Já os bancos de dados NoSQL são projetados para tolerar partições e, portanto, frequentemente priorizam disponibilidade e particionamento, aceitando a consistência eventual.

Exemplos de integração

E-commerce

Contexto: uma aplicação de e-commerce com serviços para catálogo de produtos, carrinho de compras, pedidos e pagamentos.

- **Implementação**
 - **Banco de dados por serviço:** cada serviço tem seu próprio banco de dados. O serviço de catálogo usa MongoDB para armazenar informações flexíveis sobre produtos, enquanto o serviço de pagamentos usa PostgreSQL para garantir transações seguras.
 - **Consistência eventual:** implementação de Event Sourcing para gerenciar o histórico de pedidos e garantir que todos os serviços tenham uma visão consistente dos dados.

- **CQRS:** separação das operações de comando (como adicionar um item ao carrinho) e consulta (como listar itens no carrinho) para otimizar o desempenho.

Neste exemplo, cada serviço é independente e possui seu próprio banco de dados, promovendo a autonomia e facilitando a escalabilidade. O serviço de catálogo utiliza MongoDB, um banco de dados NoSQL, para armazenar dados de produtos de forma flexível e eficiente. Já o serviço de pagamentos utiliza PostgreSQL, um banco de dados SQL, para garantir transações seguras e consistentes. A aplicação de Event Sourcing assegura que os pedidos são registrados como eventos, proporcionando um histórico completo e permitindo consistência eventual, e o uso de CQRS separa as operações de escrita e leitura, melhorando a performance e a escalabilidade do sistema.

Plataforma de Streaming

Contexto: uma plataforma de streaming com serviços para gerenciamento de usuários, biblioteca de mídia, reprodução de vídeos e recomendações.

- **Implementação**

- **Banco de dados por serviço:** o serviço de gerenciamento de usuários usa Cassandra para escalar horizontalmente e lidar com grandes volumes de dados. O serviço de recomendações usa Elasticsearch para fornecer resultados de busca rápida.
- **CQRS:** separação das operações de escrita (gerenciamento de mídia) das operações de leitura (recomendações de conteúdo) para otimizar o desempenho.
- **Event Sourcing:** cada evento de reprodução de vídeo é armazenado como um evento, permitindo a reconstrução do histórico de visualizações do usuário para oferecer recomendações personalizadas.

MERCADO, CASES E TENDÊNCIAS

Os bancos de dados em arquiteturas de microsserviços estão constantemente evoluindo para atender às crescentes demandas de escalabilidade, flexibilidade e desempenho. Diversas empresas líderes de mercado adotaram soluções inovadoras para gerenciar dados em ambientes distribuídos, ilustrando como os conceitos teóricos são aplicados na prática.

Mercado

O mercado de bancos de dados para microsserviços está dividido principalmente entre soluções SQL e NoSQL, cada uma atendendo a diferentes necessidades e casos de uso. As soluções SQL, como PostgreSQL e MySQL, continuam populares devido à sua robustez, suporte a transações ACID e facilidade de integração com sistemas legados. No entanto, o crescimento exponencial de dados e a necessidade de escalabilidade horizontal têm impulsionado a adoção de bancos de dados NoSQL, como MongoDB, Cassandra e Redis. Empresas estão cada vez mais utilizando uma combinação de ambos para aproveitar o melhor de cada tecnologia.

Case Amazon DynamoDB

Amazon DynamoDB é um exemplo notável de um banco de dados NoSQL que oferece alta escalabilidade e desempenho. Utilizado pela própria Amazon, DynamoDB é projetado para suportar cargas de trabalho intensivas e fornecer baixa latência, sendo ideal para aplicativos móveis, jogos e IoT. A capacidade de escalar automaticamente e a integração com outros serviços AWS tornam DynamoDB uma escolha popular para arquiteturas de microsserviços que exigem alta disponibilidade e consistência eventual.

Tendências

- **Banco de dados multimodelo:** uma tendência emergente é o uso de bancos de dados multimodelo que suportam vários modelos de dados (relacional, documentos, grafos etc.) em um único sistema. Isso permite às empresas utilizarem a tecnologia mais adequada para cada caso de uso

sem a complexidade de gerenciar múltiplos bancos de dados. Exemplos incluem ArangoDB e OrientDB, que combinam as capacidades de SQL e NoSQL.

- **Serverless databases:** outra tendência é a adoção de bancos de dados serverless, que eliminam a necessidade de gerenciar infraestrutura. Serviços como Amazon Aurora Serverless e Azure Cosmos DB permitem que as aplicações escalem automaticamente conforme a demanda, reduzindo a complexidade operacional e os custos. Essa abordagem é particularmente vantajosa para startups e pequenas empresas que desejam focar no desenvolvimento de produtos sem se preocupar com a gestão de servidores.
- **Inteligência artificial e aprendizado de máquina:** a integração de inteligência artificial (IA) e aprendizado de máquina (ML) em bancos de dados está ganhando tração. Ferramentas avançadas de análise preditiva e automação de processos estão sendo incorporadas para melhorar a eficiência e a tomada de decisões. Por exemplo, bancos de dados estão começando a usar IA para otimizar consultas, prever falhas de hardware e ajustar automaticamente a capacidade de acordo com os padrões de uso.
- **Blockchain:** o uso de blockchain para garantir a integridade e a transparência dos dados também está se expandindo. Aplicações que requerem um registro imutável e auditável de transações, como cadeias de suprimentos e finanças descentralizadas (DeFi), estão adotando bancos de dados baseados em blockchain. Essa tecnologia oferece uma maneira segura de registrar eventos e transações, proporcionando confiança e transparência.

Essas tendências demonstram como a inovação contínua está moldando o futuro dos bancos de dados em arquiteturas de microsserviços, permitindo que as empresas lidem com desafios complexos de maneira eficiente e eficaz. Com a evolução das necessidades empresariais e tecnológicas, a escolha e a implementação de soluções de banco de dados continuarão a desempenhar um papel crucial no sucesso dos sistemas de microsserviços.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos os desafios e soluções associados ao uso de bancos de dados em arquiteturas de microsserviços. Aprendemos sobre a importância de isolar dados por serviço e os desafios de manter a consistência e a integridade dos dados em um ambiente distribuído. Discutimos a diferença entre bancos de dados por serviço e bancos de dados compartilhados, embora a prática recomendada seja que cada serviço tenha seu próprio banco de dados.

Abordamos os padrões de design, como Event Sourcing e CQRS, que ajudam a gerenciar mudanças de estado e a separar operações de comando e consulta, respectivamente. Também analisamos as diferenças entre bancos de dados SQL e NoSQL, entendendo suas características, casos de uso e ferramentas associadas. Por fim, examinamos exemplos práticos de integração de bancos de dados em empresas líderes de mercado, como a Amazon, e discutimos o teorema CAP e sua influência na escolha de tecnologias de banco de dados.

REFERÊNCIAS

A NATIVE multi-model database. **Arangodb**, [s.d.]. Disponível em: <https://www.arangodb.com/>. Acesso em: 24 set. 2024.

AMAZON DynamoDB. **AWS**, [s.d.]. Disponível em: <https://aws.amazon.com/dynamodb/>. Acesso em: 24 set. 2024.

AZURE Cosmos DB: Globally Distributed, Multi-Model Database. **Azure**, [s.d.]. Disponível em: <https://azure.microsoft.com/en-us/services/cosmos-db/>. Acesso em: 24 set. 2024.

FOWLER, M.; LEWIS, J. Microservices: a definition of this new architecture term. **MartinFowler**, 2014. Disponível em: <https://martinfowler.com/articles/microservices.html>. Acesso em: 24 set. 2024.

NEWMAN, S. **Building Microservices. 2nd ed.** [s.l.] O'Reilly Media, 2021.

O QUE é a tecnologia blockchain? **AWS**, [s.d.]. Disponível em: <https://aws.amazon.com/pt/what-is/blockchain/>. Acesso em: 24 set. 2024.

PALAVRAS-CHAVE

Banco de dados, Arquitetura de Software, Microsserviços, CQRS, Event Sourcing, CAP, SQL, NoSQL.

EMSE

POSTECH